Algorithmie

SAÉ 3.02 - Labyrinthe

Développement efficace

Étudiants :

Victor Bredelle
Antonin Marouze
Romain Harlaut
Angèl Zheng
Baptiste Lavogiez

Enseignant :

Mme Everaere

31 octobre 2025

Table des matières

1 Introduction **2**

2 Investissement dans le projet **3**

3 Structures de données pour les labyrinthes **5**

3.1 Labyrinthes aléatoires 5

3.2 Labyrinthes parfaits 5

4 Algorithmes de génération **6**

4.1 Labyrinthes aléatoires 6

4.1.1 Algorithme utilisé 6

4.1.2 Pseudo-code 6

4.1.3 Structures de données utilisées 6

4.1.4 Pertinence 6

4.2 Labyrinthes parfaits 6

4.2.1 Algorithme utilisé 6

4.2.2 Pseudo-code 6

4.2.3 Structures de données utilisées 6

4.2.4 Pertinence 6

5 Efficacité **7**

5.1 Méthode 7

5.1.1 Test de performance 7

5.1.2 Modélisation 8

5.2 Résultats 10

5.2.1 Labyrinthes aléatoires 10

5.2.2 Labyrinthes parfaits 10

5.3 Conclusion 10

6 Complément et Bilan **11**

7 Conclusion **12**

1 Introduction

Ce rapport présente la partie algorithmique du projet **SAÉ S3.02 - Labyrinthe**, une application avec génération de divers labyrinthes.

1. Diagrammes de cas d'utilisation
2. Fiches descriptives
3. Prototype d'interface
4. Diagramme de classe

2 Investissement dans le projet

Groupe G3.3

- Victor Bredelle
- Antonin Marouze
- Romain Harlaut
- Angèl Zheng
- Baptiste Lavogiez

Nom	Victor	Antonin	Romain	Angèl	Baptiste	Total
Part	⟨Data 2.1⟩	⟨Data 2.2⟩	⟨Data 2.3⟩	⟨Data 2.4⟩	⟨Data 2.5⟩	100%

Table 2.1 – Répartition de l'investissement

1. Victor

Rédaction de trois fiches descriptives avec Baptiste : Charger profil, Afficher progression, Lancer mode progression.

2. Antonin

Réalisation de la maquette *Figma* avec Angèl et Romain.

3. Romain

Réalisation de la fiche descriptive "Déplacer joueur" et aide à la maquette *Figma*.

4. Angèl

Réalisation de la maquette *Figma* avec Antonin et Romain.

5. Baptiste

Rédaction de trois fiches descriptives avec Victor et rédaction du rapport.

Le groupe

Information

L'ensemble des contributeurs a collaboré à l'élaboration du diagramme de classes.

3 Structures de données pour les labyrinthes

3.1 Labyrinthes aléatoires

3.2 Labyrinthes parfaits

4 Algorithmes de génération

4.1 Labyrinthes aléatoires

4.1.1 Algorithme utilisé

4.1.2 Pseudo-code

pseudocode

```
toto titi
tant que toto
faire tata
```

4.1.3 Structures de données utilisées

4.1.4 Pertinence

4.2 Labyrinthes parfaits

4.2.1 Algorithme utilisé

4.2.2 Pseudo-code

pseudocode

```
toto titi
tant que toto
faire tata
```

4.2.3 Structures de données utilisées

4.2.4 Pertinence

5 Efficacité

5.1 Méthode

La présente sous-section expose notre méthodologie de manière théorique, divisée en deux parties distinctes.

1. Test de performance
2. Modélisation

5.1.1 Test de performance

On note :

- a : l'algorithme du labyrinthe,
- n : la taille initiale du labyrinthe,
- e : l'écart entre deux tests successifs,
- f : la taille maximale (fin du test).

$$\forall n > 2,\, n \in \mathbb{N}, \forall e \in \mathbb{N} :$$

pseudocode

```
créer un fichier csv de nom "a_n_e_f"
tant que n < f :
    t1 = mesurer le temps actuel
    générer un labyrinthe d'algorithme (a) de taille n (
        largeur = n, hauteur = 2*n)
    t2 = mesurer le temps actuel
    écrire sur la ligne actuelle (n, t2 - t1)
    n = n + e
    passer à la ligne suivante
```

Implémentation Java

Taille (n)	Temps de génération (ms)
10	12
15	18
20	25
25	30
30	38
35	45
⋮	⋮
495	1200
500	1250

Table 5.1 – Exemple de sortie pour $a = \text{STANDARDLARGEUR}, n = 10, e = 5, f = 500$

5.1.2 Modélisation

Ces fichiers requièrent alors une analyse par un script concis en *matplotlib*.

python

```python
import os
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from io import BytesIO

d = "res/benchmarks"
files = [f for f in os.listdir(d) if f.endswith(".csv")]
for i,f in enumerate(files): print(i,f)
i = int(input("Choix: "))

df = pd.read_csv(os.path.join(d, files[i]))

plt.plot(df["taille"], df["temps(ms)"])
plt.xlabel("taille")
plt.ylabel("temps (ms)")
plt.title(files[i])
plt.tight_layout()

```

```
buf = BytesIO()
plt.savefig(buf, format="png")
plt.show()
plt.close()

buf.seek(0)
img = Image.open(buf)
os.makedirs(d+"/modelisations", exist_ok=True)
img.save(os.path.join(d+"/modelisations", files[i].replace(
    ".csv", ".png")))
```

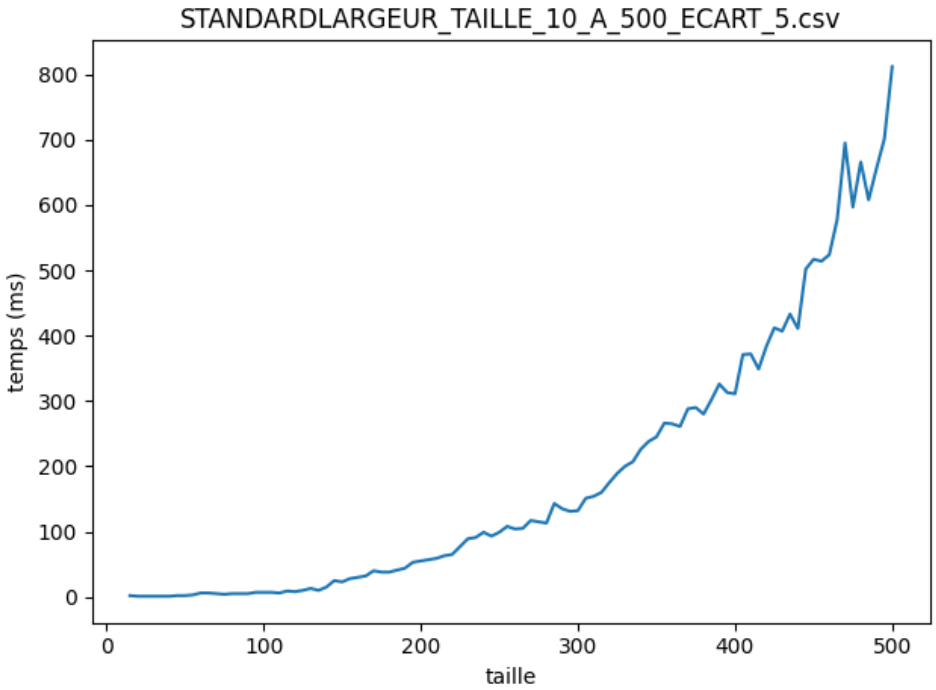

Figure 5.1 – Exemple de sortie pour $a = \text{STANDARDLARGEUR}, n = 10, e = 5, f = 500$

5.2 Résultats

5.2.1 Labyrinthes aléatoires

Taille (n)	Temps de génération (ms)
10	12
15	18
20	25
25	30
30	38
35	45
⋮	⋮
495	1200
500	1250

Table 5.2 – Exemple de sortie pour $a = \text{STANDARDLARGEUR}, n = 10, e = 5, f = 500$

5.2.2 Labyrinthes parfaits

5.3 Conclusion

6 Complément et Bilan

7 Conclusion